

# Resum sobre arrays en JavaScript

---

Aquest document és una **referència** sobre els arrays i els mètodes més habituals per treballar-hi. Cada secció inclou una explicació breu, un exemple, referències a MDN i, quan correspon, un enllaç al fitxer `.js` corresponent.

## Referència general

## Índex

- [1. Què és un array?](#)
- [2. Afegir i eliminar elements](#)
- [3. Cercar elements](#)
- [4. Funcions callback i arrow functions](#)
- [5. Arrays d'objectes i referències](#)
- [6. Buidar un array](#)
- [7. Combinar i copiar arrays](#)
- [8. Convertir un array en text](#)
- [9. Ordenar i invertir elements](#)
- [10. Comprovar condicions](#)
- [11. Filtrar elements](#)
- [12. Transformar elements amb map](#)
- [13. Acumular valors amb reduce](#)
- [14. Exercicis](#)
- [Referència general](#)

---

## 1. Què és un array?

Un **array** és una estructura que permet emmagatzemar diversos valors en una seqüència ordenada. Els índexs comencen per `0`.

```
const fruites = ["poma", "plàtan", "kiwi"];

console.log(fruites[0]); // "poma"
console.log(fruites.length); // 3
```

Els arrays són dinàmics i poden contenir valors de tipus diferents, tot i que habitualment convé que els elements tinguin una estructura coherent.

```
const barrejats = ["hola", 3, true, null];
```

## 2. Afegir i eliminar elements

`push()` i `unshift()` afegeixen elements al final i al principi. `pop()` i `shift()` eliminen i retornen l'últim i el primer element, respectivament. Aquests mètodes modifiquen l'array original.

```
const nums = [1, 2, 3];

nums.push(4);    // afegeix al final
nums.unshift(0); // afegeix al principi
nums.pop();      // elimina l'últim
nums.shift();    // elimina el primer

console.log(nums); // [1, 2, 3]
```

`splice()` pot afegir, eliminar o substituir elements en una posició concreta i també modifica l'array original.

```
const lletres = ["a", "b", "c", "d"];
lletres.splice(1, 2); // elimina dos elements des de l'índex 1
console.log(lletres); // ["a", "d"]
```

🌱 Exemples: [01-afegir.js](#) · [02-eliminar.js](#)

### Referència:

- [MDN – push\(\)](#)
- [MDN – unshift\(\)](#)
- [MDN – pop\(\)](#)
- [MDN – shift\(\)](#)
- [MDN – splice\(\)](#)

---

## 3. Cercar elements

`includes()` comprova si un valor primitiu és present. `indexOf()` retorna l'índex de la primera coincidència o `-1` si no la troba.

```
const noms = ["Anna", "Joan", "Maria"];

console.log(noms.includes("Anna")); // true
console.log(noms.indexOf("Joan"));  // 1
```

Per cercar objectes segons una condició, es poden utilitzar `find()` i `findIndex()`. Els objectes són valors de referència, així que `indexOf()` no troba un objecte diferent encara que tinguin les mateixes propietats.

```
const usuarios = [{ id: 1, nom: "Anna" }, { id: 2, nom: "Joan" }];
const usuari = usuarios.find(element => element.id === 2);

console.log(usuari); // { id: 2, nom: "Joan" }
```

✖ Exemples: [03-buscar-primitius.js](#) · [05-buscar-referencia.js](#)

### Referència:

- [MDN – includes\(\)](#)
- [MDN – indexOf\(\)](#)
- [MDN – find\(\)](#)
- [MDN – findIndex\(\)](#)

---

## 4. Funcions callback i arrow functions

Diversos mètodes d'array reben una **callback**: una funció que s'executa per a cada element o per fer una cerca. Les arrow functions (`=>`) ofereixen una sintaxi compacta per escriure aquestes funcions.

```
const doble = [1, 2, 3].map(numero => numero * 2);
console.log(doble); // [2, 4, 6]
```

✖ Exemple: [04-arrow-fuctions.js](#)

 [MDN – Arrow function expressions](#)

---

## 5. Arrays d'objectes i referències

Els arrays són objectes. Quan assignem un array o un objecte a una altra variable, es copia la referència i totes dues variables accedeixen a la mateixa estructura.

```
const original = [{ id: 1, nom: "Anna" }];
const referencia = original;

referencia[0].nom = "Joan";
console.log(original[0].nom); // "Joan"
```

Per buscar dins d'un array d'objectes, normalment es compara una propietat que identifiqui l'element, com ara `id`.

✖ Exemple: [05-buscar-referencia.js](#)

---

## 6. Buidar un array

Podem buidar un array amb `length = 0` o amb `splice()`. Assignar `[]` a una variable, en canvi, fa que aquesta variable apunti a un array nou i no buida l'array que comparteixen altres referències.

```
const dades = [1, 2, 3];
const referencia = dades;

dades.length = 0;
console.log(referencia); // []
```

🔗 Exemple: [06-buidant-arrays.js](#)

📖 [MDN – Array.length](#)

---

## 7. Combinar i copiar arrays

`concat()` combina arrays i retorna'n un de nou. L'operador spread (`...`) també permet combinar arrays o crear-ne una còpia superficial.

```
const a = [1, 2];
const b = [3, 4];

console.log(a.concat(b)); // [1, 2, 3, 4]
console.log([...a, ...b]); // [1, 2, 3, 4]
```

La còpia amb spread és superficial: si els elements són objectes niuats, les referències a aquests objectes es comparteixen.

🔗 Exemples: [07-combinant.js](#) · [08-spread.js](#)

📖 **Referència:**

- [MDN – concat\(\)](#)
  - [MDN – Spread syntax](#)
- 

## 8. Convertir un array en text

`join()` combina els elements d'un array en una cadena utilitzant el separador indicat.

```
const noms = ["Anna", "Joan", "Pau"];
console.log(noms.join(", ")); // "Anna, Joan, Pau"
```

🔗 Exemple: [09-join.js](#)

📖 [MDN – join\(\)](#)

---

## 9. Ordenar i invertir elements

`sort()` ordena l'array original. Per defecte, ordena els valors com a cadenes; per ordenar nombres cal proporcionar una funció de comparació. `reverse()` també modifica l'array original.

```
const nums = [10, 2, 5];
nums.sort((a, b) => a - b);
console.log(nums); // [2, 5, 10]

nums.reverse();
console.log(nums); // [10, 5, 2]
```

🧩 Exemple: [10-ordre.js](#)

📖 Referència:

- [MDN – sort\(\)](#)
- [MDN – reverse\(\)](#)

---

## 10. Comprovar condicions

`every()` comprova si tots els elements compleixen una condició; `some()` comprova si almenys un la compleix. Tots dos retornen un booleà.

```
const edats = [18, 22, 25];

console.log(edats.every(edat => edat >= 18)); // true
console.log(edats.some(edat => edat > 30)); // false
```

🧩 Exemple: [11-every-some.js](#)

📖 Referència:

- [MDN – every\(\)](#)
- [MDN – some\(\)](#)

---

## 11. Filtrar elements

`filter()` retorna un array nou amb els elements que compleixen la condició. No modifica l'array original.

```
const nums = [1, 2, 3, 4, 5];
const parells = nums.filter(numero => numero % 2 === 0);

console.log(parells); // [2, 4]
```

 Exemple: [12-filter.js](#)

 [MDN – filter\(\)](#)

---

## 12. Transformar elements amb `map()`

`map()` aplica una funció a cada element i retorna un array nou amb els resultats.

```
const noms = ["anna", "joan", "maria"];
const majuscles = noms.map(nom => nom.toUpperCase());

console.log(majuscles); // ["ANNA", "JOAN", "MARIA"]
```

 Exemple: [13-map.js](#)

 [MDN – map\(\)](#)

---

## 13. Acumular valors amb `reduce()`

`reduce()` recorre l'array i acumula els elements en un únic resultat. El valor inicial de l'acumulador s'indica com a segon argument.

```
const nums = [1, 2, 3, 4];
const suma = nums.reduce((acumulador, numero) => acumulador + numero, 0);

console.log(suma); // 10
```

 Exemple: [14-reduce.js](#)

 [MDN – reduce\(\)](#)

---

## 14. Exercicis

Els exercicis permeten practicar la cerca, la transformació, el filtratge i l'ordenació d'arrays.

 [Fitxers d'exercicis](#)

---

## Referència general

- [MDN – Array](#)
- [MDN – Indexed collections](#)